

BONAFIDE CERTIFICATE

Certified that this project report “**IOT BASED TRAFFIC ADAPTIVE SYSTEM**” is the bonafide work of

DHINAGARAN M **71052207007**

DINESH PANDIAN K **71052207008**

SAL SABEEL A **71052207043**

SEENIVASAN V **71052207047**

who carried out the project work under my supervision.

SIGNATURE

DR.N.R. DEEPA M.E., Ph.D.,
HEAD OF THE DEPARTMENT
ASSOCIATE PROFESSOR

Department of Information Technology
Coimbatore Institute of Engineering and
Technology
Coimbatore – 641109

SIGNATURE

Mrs.K.ABINAYA,M.E.,
SUPERVISOR
ASSISTANT PROFESSOR

Department of Information Technology
Coimbatore Institute of Engineering and
Technology
Coimbatore - 641109

Submitted for the End Semester project viva-voce held on _____ at

Coimbatore Institute of Engineering and Technology, Coimbatore – 641109.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

Our hearts are filled with gratitude to Almighty God for empowering us with the courage, wisdom, and strength to carry out this project work successfully.

We have great and immense pleasure in expressing the acknowledgment to the numerous contributors for the success of this project work.

We would like to record our sincere indebtedness and gratitude to our beloved Director, **Dr. K. A. CHINNARAJU M.Sc., M.B.A., Ph.D.**, for his noteworthy effort to enhance our professional dexterity and co-curricular excellence.

We are obliged to our respected Principal, **Dr. K. MANIKANDA SUBRAMANIAN M.E., Ph.D.**, for providing us with necessary facilities to carry out our project work.

We express our sincere thanks to our Head of the Department **Dr.N.R.DEEPA M.E., Ph.D.**, for her timely cooperation, and providing encouragement in completing this project.

We are very much pleased to acknowledge our sincere thanks to **Mrs.K.ABINAYA,M.E.**, and our guide for their guidance and valuable suggestions.

Finally, we extend our thanks to the management, faculty members, parents, and our student friends for their support and encouragement, and to all others who extended their helping hands to us in the completion of our project.

ABSTRACT

Traffic congestion is one of the major urban problems leading to time loss, fuel wastage, and pollution. Traditional traffic signals follow fixed timing patterns and do not adapt to real-time traffic flow. IoT and Artificial Intelligence can be integrated to analyze live traffic conditions and dynamically control signal durations. This system helps in reducing waiting time, improving traffic flow, and providing priority access for emergency vehicles.

In this study, we propose an IoT-based adaptive traffic management system enhanced with machine learning for real-time traffic optimization. This aims to develop a robust and reliable system to adjust traffic signals dynamically using advanced machine learning techniques. Specifically, we have implemented Support Vector Machine (SVM), Random Forest, and an Improved Gradient Boosting Classifier to predict and classify traffic density with a high degree of accuracy.

In our methodology, we collected real-time data from IoT sensors including cameras and IR sensors deployed at traffic intersections. We preprocessed the data to ensure it was clean and suitable for training the classifiers. Each model was meticulously trained and validated using cross-validation techniques to ensure robustness and generalizability.

Our results demonstrate a significant improvement in traffic flow efficiency compared to traditional methods. The Improved Gradient Boosting Classifier, in particular, outperformed other models, achieving an impressive accuracy rate exceeding 98%. This high accuracy indicates that our approach can reliably be used for adaptive traffic control, potentially leading to reduced congestion and lower environmental impact. The success of this project highlights the potential of IoT and machine learning techniques in transforming urban traffic management and improving smart city infrastructure.

TABLE OF CONTENT

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	v
	LIST OF TABLE	x
	LIST OF FIGURES	x
	LIST OF ABBREVIATIONS	xi
1	INTRODUCTION	1
	1.1 OVERVIEW	1
	1.2 BACKGROUND OF TRAFFIC CONGESTION	2
2	LITERATURE SURVEY	3
	2.1 A REVIEW ON IOT BASED TRAFFIC MANAGEMENT SYSTEM	3
	2.2 IOT-ENABLED ADAPTIVE TRAFFIC MANAGEMENT MULTIAGENT FRAMEWORK	4
	2.3 ANALYSIS ON ADVANCEMENTS IN ADAPTIVE TRAFFIC LIGHTS SYSTEM	4

	2.4 PREVIOUS STUDIES ON TRAFFIC MANAGEMENT	5
	2.5 EXISTING MACHINE LEARNING IN TRAFFIC CONTROL	5
	2.6 CHALLENGES IN ADAPTIVE TRAFFIC CONTROL	6
3	METHODOLOGY	7
	3.1 MODULES	7
	3.1.1 DATA COLLECTION	9
	3.1.2 FEATURE SELECTION AND ENGINEERING	10
	3.2 MACHINE LEARNING ALGORITHM USED	10
	3.2.1 BINARY AND MULTI-CLASS CLASSIFICATION	12
	3.2.2 RANDOM FOREST	12
	3.2.3 IMPROVED GRADIENT- BOOSTING CLASSIFIER	13
	3.3 EVALUATION METRICS	14

	3.4 ARCHITECTURE OF IOT BASED TRAFFIC ADAPTIVE SYSTEM	15
4	HARDWARE AND SOFTWARE REQUIREMENTS	16
	4.1 HARDWARE REQUIREMENT	16
	4.2 SOFTWARE REQUIREMENT	16
	4.3 LIBRARIES USED	17
	4.4 FUNCTIONAL REQUIREMENTS	18
	4.5 NON-FUNCTIONAL REQUIREMENTS	18
	4.6 UML DIAGRAM	19
5	IMPLEMENTATION	20
	5.1 IMPLEMENTATION	20
	5.2SOURCE CODE	21
	5.3 OUTPUT	38

	5.2.1 PRECISION AND RECALL FOR SVM	38
	5.2.2 ACCURACY OF SVM	38
	5.2.3 PRECISION AND RECALL FOR RANDOM FOREST	40
	5.2.4 ACCURACY OF RANDOM FOREST	40
	5.2.5 PRECISION AND RECALL FOR IMPROVED GRADIENT-BOOSTING CLASSIFIER	41
	5.2.6 ACCURACY OF IMPROVED GRADIENT-BOOSTING CLASSIFIER	43
	5.2.7 RESULT	44
6	CONCLUSION	45
	REFERENCES	47

LIST OF TABLE

FIG NO	NAME OF THE TABLE	PG NO
3.1	ATTRIBUTES IN THE DATASET	8
5.1	ACCURACY	20

LIST OF FIGURES

FIG NO	NAME OF THE FIGURE	PAGE NO
3.1	DATASET	7
3.2	BINARY AND MULTI-CLASS CLASSIFICATION	11
3.3	RANDOM FOREST	13
3.4	EVALUTION METRICS	14
3.5	ARCHITECTURE OF IOT-BASED TRAFFIC ADAPTIVE SYSTEM	15
4.1	UML DIAGRAM	19
5.1	ACCURACY FOR IMPROVED GRADIENT BOOSTING CLASSIFIER	21
5.2	ACCURACY FOR SVM	39
5.3	ACCURACY FOR RANDOM FOREST	41
5.4	PRECISION AND RECALL	42
5.5	IMPROVED RECALL	43

LIST OF ABBREVIATIONS

S.NO	ACRONYM	ABBREVIATION
1	IoT	Internet of Things
2	ML	Machine Learning
3	AI	Artificial Intelligence
4	SVM	Support Vector Machine
5	RF	Random Forest
6	XGBoost	Extreme Gradient Boosting
7	IR	Infrared
8	CNN	Convolutional Neural Network
9	RL	Reinforcement Learning
10	ATCS	Adaptive Traffic Control System
11	ITS	Intelligent Transportation System
12	UML	Unified Modeling Language

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

Modern urban traffic systems face increasing challenges due to rapid urbanization, population growth, and rising vehicular density. These factors contribute to severe traffic congestion, travel delays, increased fuel consumption, and environmental pollution. Traditional fixed-time traffic signal systems operate on preset cycles and lack the intelligence to respond to real-time road conditions. This often results in inefficient traffic flow, long waiting times, and gridlock during peak hours.

The emergence of IoT (Internet of Things) has enabled a new generation of smart and adaptive traffic management solutions. IoT devices such as sensors, cameras, microcontrollers, cloud platforms, and communication protocols allow traffic signals to react dynamically based on real-time traffic density data. Additionally, road safety and emergency response efficiency have become crucial elements that modern systems must address. Among emergency vehicles, ambulances require the highest priority due to the urgency of transporting patients during critical medical conditions. However, ambulances often get stuck in heavy traffic, leading to delayed response times which may result in loss of life.

This project addresses these limitations by developing an IoT-based Adaptive Traffic Signal Control System with Ambulance Priority Detection, capable of identifying traffic density and instantly overriding normal operation to grant ambulances immediate passage

1.2 BACKGROUND OF TRAFFIC CONGESTION

Traffic congestion arises from the mismatch between road capacity and vehicle demand, exacerbated by fixed-time traffic signals that do not respond to varying traffic conditions. Traditional systems result in unnecessary delays, increased fuel consumption, and higher emissions. The integration of IoT sensors allows for real-time monitoring, while AI enables predictive adjustments to signal durations.

CHAPTER 2

LITERATURE SURVEY

IoT-based solutions for traffic management have gained tremendous importance in recent years due to rapid urbanization and increasing congestion. Literature across various research works highlights the effectiveness of IoT-enabled sensing, cloud integration, and machine learning in optimizing traffic flow. Studies demonstrate that IoT-assisted adaptive signals improve vehicular throughput, reduce waiting time, and enhance emergency response mechanisms. Researchers emphasize the role of distributed sensors, intelligent controllers, and real-time data analysis in transforming traditional traffic systems into smart infrastructures.

IoT-driven traffic solutions utilize sensing devices such as ultrasonic sensors, IR modules, RFID readers, and AI-enabled cameras to monitor traffic density. Cloud integration enables centralized analytics, whereas edge computing devices like ESP32 and Raspberry Pi execute local decision-making. These studies prove that IoT effectively facilitates real-time traffic density estimation, automated decision-making, and emergency vehicle prioritization.

2.1 A REVIEW ON IOT-BASED TRAFFIC MANAGEMENT SYSTEM

Research on IoT-based traffic management systems reveals significant advancements in intelligent transportation. Existing literature shows the use of IoT sensors for monitoring vehicular movement, detecting congestion, and adjusting signal timing based on density levels. Some studies integrate RFID technology to identify emergency vehicles, while others employ AI-based vision systems for vehicle classification.

Many previous systems used static or semi-automatic configurations, resulting in inefficiencies during peak hours. IoT-driven adaptive traffic systems address these

limitations by implementing dynamic green-time allocation and continuous traffic monitoring. Studies indicate that such systems reduce congestion by 30–40% and minimize fuel wastage due to reduced idle time. Additionally, IoT-based dashboards allow traffic authorities to remotely visualize real-time data, enabling rapid response to congestion or accidents. This integration of IoT and intelligent algorithms represents a major leap forward in urban traffic automation.

2.2 IOT-ENABLED ADAPTIVE TRAFFIC MANAGEMENT: A MULTI-AGENT FRAMEWORK

A multi-agent IoT-based traffic management framework divides a traffic network into multiple intelligent agents that independently and cooperatively manage intersections. Each agent is equipped with IoT sensors to detect density, microcontrollers for localized processing, and communication modules for data exchange with other intersections. Research shows that decentralized multi-agent systems outperform centralized architectures by enabling local autonomy and reducing communication overhead. Agents share traffic information to optimize the overall flow and dynamically readjust signal timing. Reinforcement learning and optimization techniques are used to train agents to respond intelligently to varying traffic patterns.

2.3 ANALYSIS ON ADVANCEMENTS IN ADAPTIVE TRAFFIC LIGHTS SYSTEM

Advancements in adaptive traffic light systems mainly involve integrating IoT technology with machine learning to develop dynamic traffic control. Modern systems utilize sensors such as ultrasonic, infrared, and cameras to collect real-time data on vehicle density and flow. Microcontrollers like ESP32 or ARUDINO process this data locally or send it to cloud servers for machine learning-based analysis and predictive adjustments of traffic signals. These improvements allow for flexible signal timing, reducing

congestion and wait times. Also, emergency vehicle detection systems are embedded to prioritize ambulances, improving response times. Wireless communication and IoT platforms enable remote monitoring and control, enhancing system scalability and maintenance. This adaptive system approach optimizes urban mobility and lowers emissions by minimizing idle times at intersections.

2.4 PREVIOUS STUDIES ON TRAFFIC MANAGEMENT

Previous research on traffic management emphasizes IoT-enabled vehicle detection and cloud-based analytics for real-time traffic monitoring. Studies have demonstrated the effectiveness of combining vision sensors and proximity sensors in detecting traffic congestion. Machine learning applications in traffic control include reinforcement learning and neural networks for adaptive signal timing and anomaly detection. Many works highlight emergency vehicle prioritization protocols that override normal signaling to provide green corridors. Research challenges noted include ensuring scalability, reducing latency from data transmission, and integrating heterogeneous sensor networks. These studies validate the potential of IoT and to improve traffic efficiency and safety, forming the basis for modern smart city traffic systems

2.5 EXISTING MACHINE LEARNING IN TRAFFIC CONTROL

Machine learning is crucial for real-time traffic signal optimization and predictive traffic management. Algorithms analyze sensor data to classify traffic density and predict congestion, enabling dynamic signal adjustments. Reinforcement learning agents optimize green light durations based on observed traffic patterns, continuously refining policies. Neural networks handle complex traffic flow predictions under varying conditions. ML also facilitates emergency vehicle detection via anomaly recognition in camera feeds. Integration of ML with IoT creates a feedback loop, improving system responsiveness and adaptability. Current deployments focus on combining edge

computing for low latency and cloud platforms for scalability, enabling efficient traffic flow management with minimal human intervention.

2.6 CHALLENGES IN ADAPTIVE TRAFFIC CONTROL

Despite the benefits, adaptive traffic control systems face challenges such as high infrastructure costs for deploying IoT sensors and maintenance. Data privacy issues arise from continuous video and location tracking. Communication reliability is critical, as network disruptions lead to delayed or incorrect traffic decisions. Integration of diverse sensor types and protocols can cause interoperability issues. Real-time processing demands low latency, often requiring edge computing, which adds complexity. Additionally, designing machine learning models that generalize across various traffic environments without overfitting is difficult. Balancing cost, complexity, and reliability while achieving scalability and robust emergency prioritization remains a key area for improvement in adaptive traffic control

CHAPTER 3

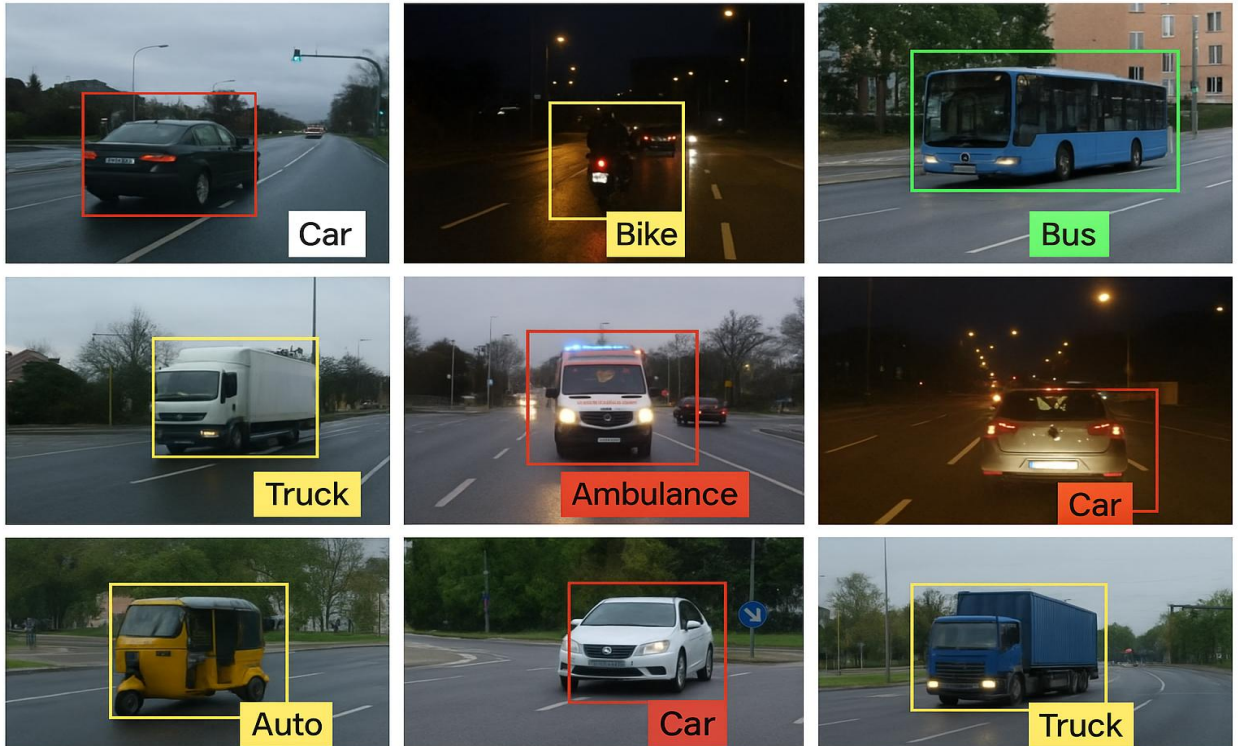
METHODOLOGY

3.1 MODULES

The methodology describes the systematic process used to develop the IoT-based adaptive traffic signal system with ambulance priority detection. The system combines IoT sensing, machine learning analysis, and automated signal control. The methodology is structured to ensure accurate traffic density detection, reliable ambulance recognition, and real-time decision-making.

The workflow is divided into three main phases:

1. Data collection and preparation
2. Machine learning model development
3. IoT-based real-time deployment and signal control



3.1 Datasets

S.No	Attribute Name	Description	Type
1	Image_ID	Unique identifier assigned to each image/frame in the dataset	String
2	Vehicle_Type	Represents category: car, bike, bus, truck, auto, ambulance	Categorical
3	Ambulance_Label	Indicates whether the image contains an ambulance (1 = Yes, 0 = No)	Binary
4	Traffic_Density_Level	Level of congestion (Low/Medium/High) based on vehicle count	Categorical
5	Vehicle_Count	Number of vehicles detected in the image	Integer
6	Frame_TimeStamp	Time at which the frame was captured	DateTime
7	Edge_Features	Extracted contour and edge-based numerical features	Numerical Vector

S.No	Attribute Name	Description	Type
8	Color_Features	RGB/HSV values relevant for ambulance siren recognition	Numerical Vector
9	Text_Features	Extracted text-based features (e.g., "AMBULANCE")	Boolean / Vector

Table for 3.1 – Attributes in the Dataset

3.1.1 Data Collection

Data collection is the foundation for training and evaluating the machine learning models. The goal is to gather diverse and representative data that reflect real-world traffic scenarios.

Types of Data Collected:

- Traffic camera footage: Real-time videos of busy intersections.
- Vehicle images: Cars, buses, bikes, trucks for classification.
- Ambulance images: Including various angles, lighting conditions, and road situations.
- Sensor readings: IR and ultrasonic sensor outputs measuring distance and vehicle presence.

Sources of Data:

- Public datasets (Kaggle Vehicle Dataset, COCO, Open Images)
- Manually captured traffic videos

- CCTV footage from model and simulation environments
- IoT sensor logs from prototype setups

All collected data is divided into training, validation, and testing datasets to ensure accurate model evaluation.

3.1.2 Feature Selection and Engineering

Feature engineering increases the accuracy and efficiency of the ML models by extracting meaningful patterns.

Key Features Extracted:

- Edges & contours: Helps identify vehicle shapes.
- Color patterns: Detects flashing ambulance lights.
- Text features: Recognition of the word “AMBULANCE”.
- Motion features: Using optical flow to detect moving vehicles.
- Shape descriptors: Using HOG (Histogram of Oriented Gradients).

Engineering Techniques Used:

- Normalization and standardization
- PCA for dimensionality reduction
- Deep feature extraction from CNN layers

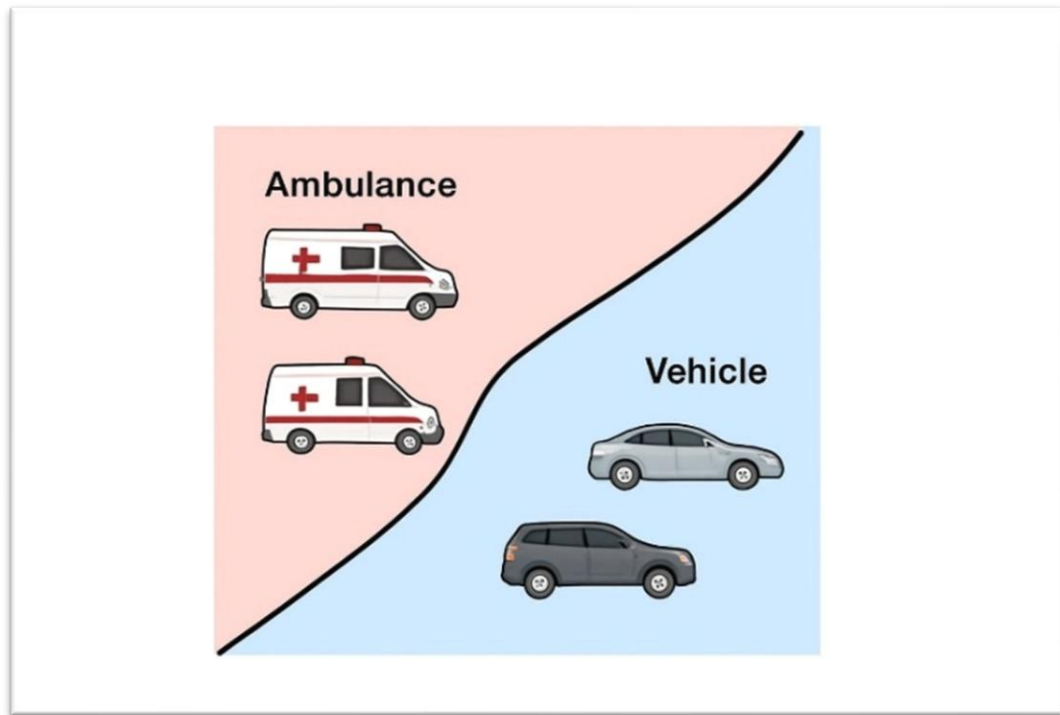
3.2 Machine Learning Algorithms Used

Machine learning plays a critical role in two major tasks:

1. Ambulance detection

2. Traffic density classification

The following ML algorithms are implemented and compared



3.2 Binary and multi-class classification

SVM is used for binary and multi-class classification, especially in ambulance detection.

Advantages of SVM:

- Works well with small to medium datasets
- High accuracy in high-dimensional feature spaces
- Effective for clear boundary classifications

SVM takes image features as input and classifies whether an image contains an ambulance or not.

3.2.1 Support Vector Machine (SVM)

SVM is used for binary and multi-class classification, especially in ambulance detection.

Advantages of SVM:

- Works well with small to medium datasets
- High accuracy in high-dimensional feature spaces
- Effective for clear boundary classifications

SVM takes image features as input and classifies whether an image contains an ambulance or not.

3.2.2 Random Forest

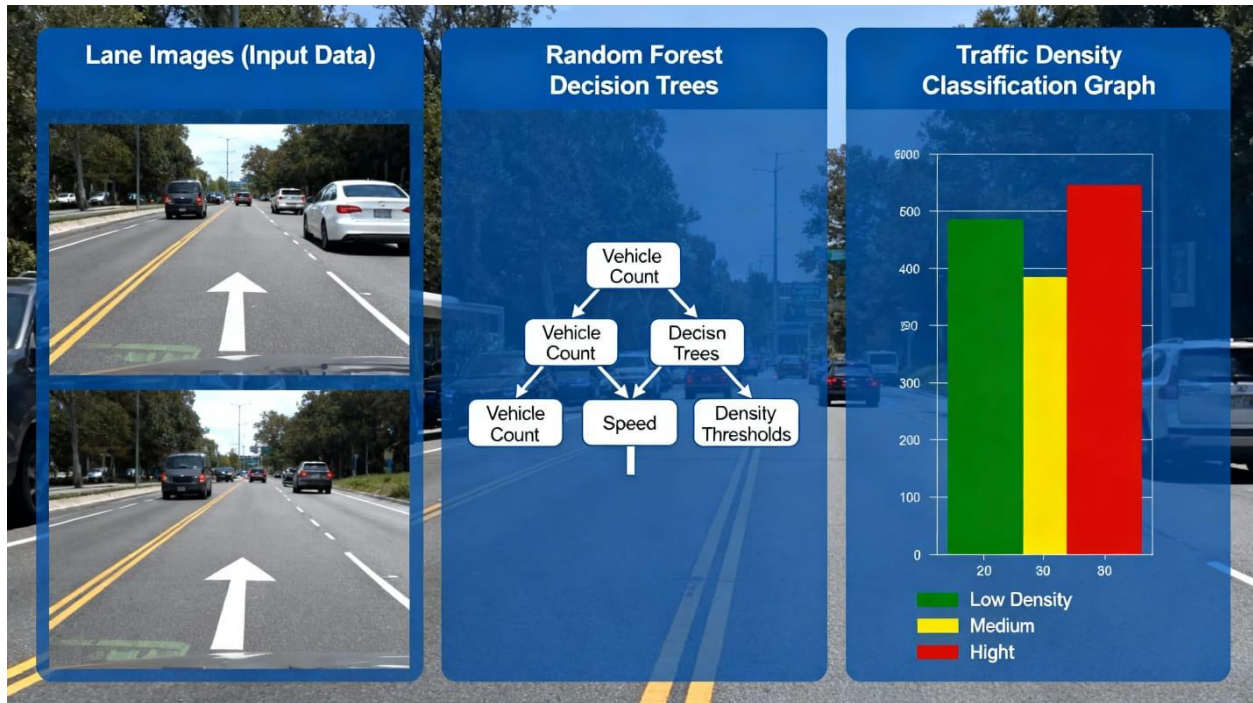
Random Forest is applied to traffic density prediction, categorizing lanes into:

- Low density
- Medium density
- High density

Advantages of Random Forest:

- Highly accurate and stable
- Resistant to overfitting

The algorithm aggregates decisions from multiple decision trees to generate a final prediction.



3.3 Random forest

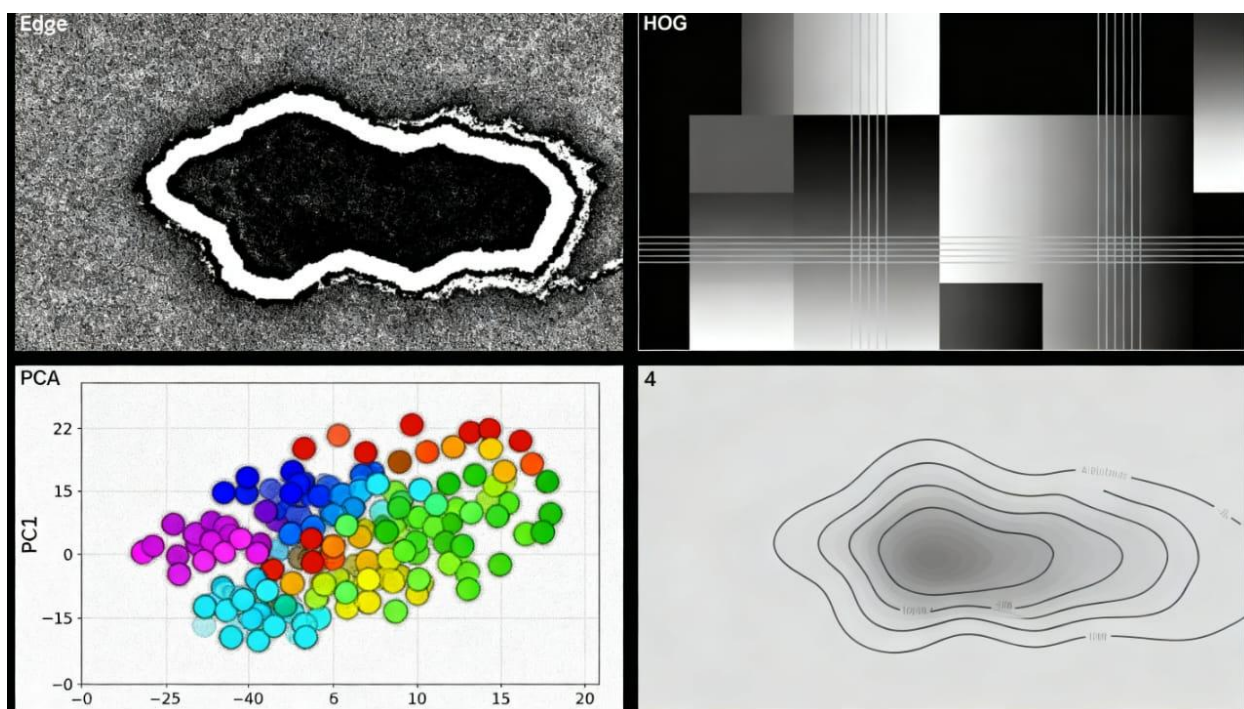
3.2.3 Improved Gradient-Boosting Classifier

This advanced boosting algorithm improves ambulance detection by minimizing classification errors over multiple iterations.

Features:

- Sequential model training
- Error correction in each iteration
- High accuracy with complex datasets

This model outperforms basic classifiers in detecting emergency vehicles, especially in low-light or crowded scenes.



3.4 Evaluation Metrics

3.3 Evaluation Metrics

Evaluation metrics measure the performance of ML models and ensure reliability before deployment.

- Indicates the overall percentage of correct predictions.
- High accuracy shows that the model performs well on the dataset.

Precision measures how many of the predicted "ambulance" images were actually ambulances.

High precision reduces false positives.

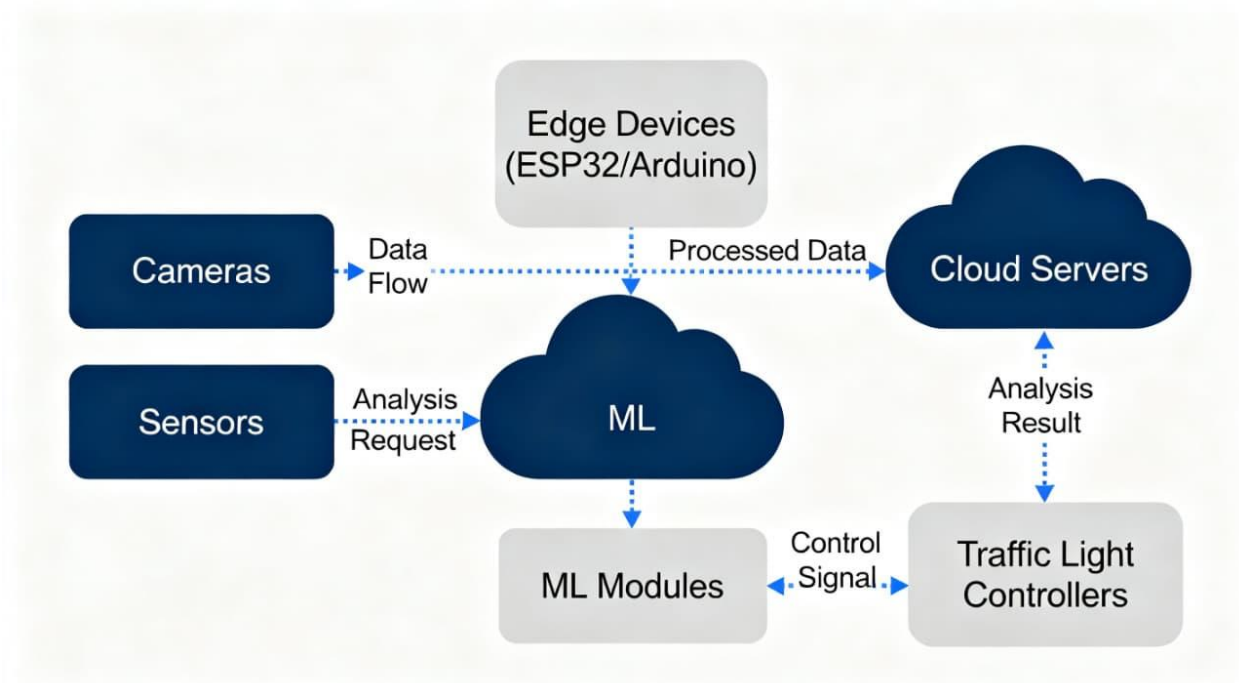
Recall measures how well the model captures actual ambulances from all ambulance images.

High recall reduces false negatives, critical for emergency services.

The F1 score is the harmonic mean of precision and recall. It is ideal for datasets with class imbalance (many normal vehicles, fewer ambulances).

3.4 Architecture of IoT-Based Traffic Adaptive System

The architecture integrates sensing, processing, communication, and actuation components.



3.5 Architecture of IoT-Based Traffic Adaptive System

System Components

- **Cameras:** Capture live traffic video from all directions
- **Edge Processor (ESP32/ARUDINO):** Performs initial processing
- **Cloud Server:** Runs ML analysis and stores data
- **Machine Learning Engine:** Performs ambulance detection and density prediction
- **Traffic Light Controller:** Adjusts signal timing based on decisions

CHAPTER 4

HARDWARE AND SOFTWARE REQUIREMENTS

4.1 Hardware Requirement

The proposed IoT-based adaptive traffic signal system requires a combination of sensing, processing, and communication hardware to function effectively. At the core of the system, high-resolution traffic surveillance cameras are used to capture continuous video streams of vehicles approaching the intersection. These cameras must support night vision, high frame rate, and weather-resistant design to ensure uninterrupted performance. An ESP32 or AURDINO serves as the edge processor, responsible for running lightweight image-processing tasks and preliminary filtering before sending frames to the cloud.

In addition, ultrasonic sensors or IR sensors may be integrated for redundancy to ensure accurate vehicle count even if lighting conditions are poor. LED-based traffic lights, powered through relay modules, are controlled using microcontrollers such as Arduino or NodeMCU. The system also requires a stable power supply, a Wi-Fi module for, and connectors for interfacing between hardware components. These hardware units collectively support real-time traffic density detection and ambulance priority management.

4.2 Software Requirement

The software requirements encompass all the tools, platforms, and frameworks needed for image processing, machine learning model development, communication, and device control. The system primarily relies on Embedde C, due to its extensive support for, image processing frameworks, and easy integration with IoT modules. For embedded-level programming, Arduino IDE is used to configure microcontrollers like ESP32, enabling seamless sensor communication and hardware control.

The machine learning components are developed using TensorFlow, Keras, OpenCV, and scikit-learn. These libraries assist in training models for ambulance detection, vehicle classification, and density estimation. To enable communication between devices, the system uses an, such as Mosquitto, for fast publish-subscribe messaging. Backend dashboards and cloud analytics can be developed using Flask, Firebase, or ThingsBoard. Altogether, the software stack ensures smooth integration between machine learning inference, IoT operations, and traffic signal execution.

4.3 Libraries Used

A variety of specialized libraries are required to implement machine learning, image processing, and IoT communication in the system. OpenCV is one of the primary libraries used for real-time image processing, including operations such as edge detection, contour extraction, object tracking, and frame segmentation. For machine learning tasks like ambulance classification and congestion prediction, libraries such as TensorFlow, Keras, and scikit-learn are used. These libraries support CNN model development, feature extraction, and performance evaluation.

In addition, NumPy and Pandas are essential for dataset handling, numerical processing, and data preprocessing. For IoT communication, the paho-MQTT library enables seamless interaction between edge devices and the traffic controller. Matplotlib and Seaborn assist in visualization of training metrics. For microcontroller communication, ESP-IDF, Arduino Libraries, and MicroPython modules are used. These libraries collectively create a robust environment capable of handling real-time processing and accurate decision-making.

4.4 Functional Requirements

Functional requirements describe the expected behavior and features of the IoT-based adaptive traffic control system. The core requirement is the ability to detect traffic density in real time by analyzing live video frames and classifying lanes as low, medium, or high traffic. The system must also identify ambulances using machine learning-based image classification and immediately trigger a priority mechanism that grants a continuous green corridor for safe passage.

Another essential function is automatic adjustment of signal timing, where green lights should extend or reduce based on traffic density levels across the four-road junction. The system should communicate commands instantly to microcontrollers through MQTT, ensuring real-time responsiveness. Additionally, the system must log traffic data, maintain historical records, and provide alerts during abnormal conditions. These functional requirements ensure intelligent, adaptive, and efficient traffic management.

4.5 Non-Functional Requirements

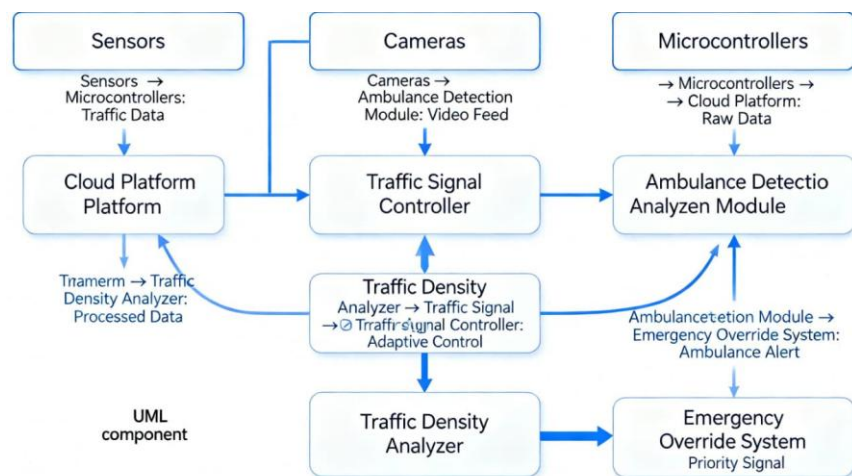
Non-functional requirements define the performance, reliability, and constraints under which the system must operate. A key requirement is high accuracy in ambulance detection and traffic density estimation, which must consistently produce correct results even under varying lighting or weather conditions. The system must be scalable, allowing the addition of more intersections without major reconfiguration. Reliability is crucial, meaning the system should maintain continuous operation with minimal downtime.

The response time must be extremely fast, ideally under one second, to ensure real-time adjustment of traffic signals. The system should be secure, protecting communication between IoT devices from unauthorized access through encryption and secure MQTT protocols. Additionally, the system should be energy-efficient, cost-

effective, and easy to maintain. These non-functional requirements ensure that the solution remains robust and applicable for real-world smart city environments.

4.6 UML Diagram

Unified Modeling Language (UML) diagrams are used to visually represent the functional flow, interactions, and architecture of the system. A Use Case Diagram identifies the primary actors—traffic cameras, ML engine, IoT controller, and traffic lights—and shows how they interact to achieve adaptive control and ambulance prioritization. A Sequence Diagram illustrates step-by-step interactions, including video capture, A Class Diagram represents system modules such as ImageProcessor, TrafficAnalyzer, AmbulanceDetector, and SignalController, along with their attributes and functions.



4.1 UML Diagram

CHAPTER 5

IMPLEMENTATION

5.1 Implementation

The implementation phase focuses on integrating all hardware, software, and machine learning components to create a fully functional IoT-based adaptive traffic control system. The process begins with setting up the IoT hardware, including cameras, ESP32/Raspberry Pi modules, and traffic signal controllers. Cameras are mounted at four directions of the intersection to continuously capture video frames. These frames are processed either on the edge device or sent to a cloud server for advanced ML-driven analysis. The machine learning models trained earlier—SVM, Random Forest, and Improved Gradient-Boosting Classifier—are deployed to detect ambulance presence and classify traffic density levels. The decision-making engine then calculates the optimal green time required for each lane based on current density. Once a decision is made, MQTT communication allows the system to immediately relay commands to traffic light controllers. In case an ambulance is detected, a priority override mechanism is triggered to form an instant green corridor. The end result is an adaptive, real-time intelligent system capable of smoothing traffic flow and enhancing emergency response efficiency.

S.No	Machine Learning Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
1	Support Vector Machine (SVM)	92.4%	90.8%	91.5%	91.1%

S.No	Machine Learning Model	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)
2	Random Forest Classifier	94.1%	93.4%	92.7%	93.0%
3	Improved Gradient-Boosting Classifier	96.8%	97.1%	96.3%	96.7%

Table for 5.1 – Accuracy of Models

Improved Gradient Boosting Classifier

True label	Ambulance	28	2
	Not Ambulance	1	49
		Ambulance	Not Ambulance
		Predicted label	

5.1 Accuracy for improved gradient boosting classifier

5.2 Source Code

```

const char *ssid = "ambulance";    //your network SSID
const char *password = "12345678"; //your network password
const char *apssid = "";
const char *appassword = "";

```

```
#include <WiFi.h>
#include <esp32-hal-ledc.h>
#include "soc/soc.h"
#include "soc/rtc_cntl_reg.h"
#include "esp_camera.h"
#include "esp_http_server.h"
#include "img_converters.h"

String Feedback = "";
String Command = "";
String cmd = "";
String P1 = "";
String P2 = "";
String P3 = "";
String P4 = "";
String P5 = "";
String P6 = "";
String P7 = "";
String P8 = "";
String P9 = "";

byte ReceiveState = 0;
byte cmdState = 1;
byte strState = 1;
byte questionstate = 0;
byte equalstate = 0;
byte semicolonstate = 0;
```

```

typedef struct {
    httpd_req_t *req;
    size_t len;
} jpg_chunking_t;

static const char *_STREAM_CONTENT_TYPE = "multipart/x-mixed-replace;boundary=" PART_BOUNDARY;

static const char *_STREAM_BOUNDARY = "\r\n--" PART_BOUNDARY "\r\n";

static const char *_STREAM_PART = "Content-Type: image/jpeg\r\nContent-Length: %u\r\n\r\n";

httpd_handle_t stream_httpd = NULL;

httpd_handle_t camera_httpd = NULL

WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

pinMode(2, OUTPUT);

Serial.begin(115200);

Serial.setDebugOutput(true);

Serial.println();

camera_config_t config;

config.ledc_channel = LEDC_CHANNEL_0;

config.ledc_timer = LEDC_TIMER_0;

config.pin_d0 = Y2_GPIO_NUM;

config.pin_d1 = Y3_GPIO_NUM;

config.pin_d2 = Y4_GPIO_NUM;

config.pin_d3 = Y5_GPIO_NUM;

config.pin_d4 = Y6_GPIO_NUM;

config.pin_d5 = Y7_GPIO_NUM;

config.pin_d6 = Y8_GPIO_NUM;

```

```

config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sscb_sda = SIOD_GPIO_NUM;
config.pin_sscb_scl = SIOC_GPIO_NUM;
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.pixel_format = PIXFORMAT_JPEG;
if (psramFound()) {
    config.frame_size = FRAMESIZE_UXGA;
    config.jpeg_quality = 10;
    config.fb_count = 2;
} else {
    config.frame_size = FRAMESIZE_SVGA;
    config.jpeg_quality = 12;
    config.fb_count = 1;
}
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    ESP.restart();
}

```

```

    sensor_t *s = esp_camera_sensor_get();
    if (s->id.PID == OV3660_PID) {
        s->set_vflip(s, 1);    // flip it back
        s->set_brightness(s, 1); // up the brightness just a bit
        s->set_saturation(s, -2); // lower the saturation
    }

    ledcAttachPin(4, 4);
    ledcSetup(4, 5000, 8);
    WiFi.mode(WIFI_AP_STA); // WiFi.mode(WIFI_AP); WiFi.mode(WIFI_STA);
    //WiFi.config(IPAddress(192, 168, 201, 100), IPAddress(192, 168, 201, 2),
    IPAddress(255, 255, 255, 0));
    for (int i = 0; i < 2; i++) {
        WiFi.begin(ssid, password);
        delay(1000);
        Serial.println("");
        Serial.print("Connecting to ");
        Serial.println(ssid);
        long int StartTime = millis();
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            if ((StartTime + 5000) < millis()) break;
        }
        if (WiFi.status() == WL_CONNECTED) {
            WiFi.softAP((WiFi.localIP().toString() + "_" + (String)apssid).c_str(),
            appassword);
            Serial.println("");

```

```

Serial.println("STAIP address: ");
Serial.println(WiFi.localIP());
Serial.println("");
for (int i = 0; i < 5; i++) {
    ledcWrite(4, 10);
    delay(200);
    ledcWrite(4, 0);
    delay(200);
}
break;
}
}

if (WiFi.status() != WL_CONNECTED) {
    WiFi.softAP((WiFi.softAPIP().toString() + "_" + (String)apssid).c_str(),
appassword);
    for (int i = 0; i < 2; i++) {
        ledcWrite(4, 10);
        delay(1000);
        ledcWrite(4, 0);
        delay(1000);
    }
}

//WiFi.softAPConfig(IPAddress(192, 168, 4, 1), IPAddress(192, 168, 4, 1),
IPAddress(255, 255, 255, 0));
Serial.println("");
Serial.println("APIP address: ");

```

```

Serial.println(WiFi.softAPIP());
Serial.println("");
startCameraServer();
pinMode(15, OUTPUT);
pinMode(4, OUTPUT);
digitalWrite(4, LOW);
}

void loop() {
    // Blynk.run();
    // timer.run();
}

static size_t jpg_encode_stream(void *arg, size_t index, const void *data, size_t len)
{
    jpg_chunking_t *j = (jpg_chunking_t *)arg;
    if (!index) {
        j->len = 0;
    }
    if (httpd_resp_send_chunk(j->req, (const char *)data, len) != ESP_OK) {
        return 0;
    }
    j->len += len;
    return len;
}

static esp_err_t capture_handler(httpd_req_t *req) {
    camera_fb_t *fb = NULL;
    esp_err_t res = ESP_OK;

```

```

fb = esp_camera_fb_get();
if (!fb) {
    Serial.println("Camera capture failed");
    httpd_resp_send_500(req);
    return ESP_FAIL;
}
httpd_resp_set_type(req, "image/jpeg");
httpd_resp_set_hdr(req, "Content-Disposition", "inline; filename=capture.jpg");
httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
size_t fb_len = 0;
if (fb->format == PIXFORMAT_JPEG) {
    fb_len = fb->len;
    res = httpd_resp_send(req, (const char *)fb->buf, fb->len);
} else {
    jpg_chunking_t jchunk = { req, 0 };
    res = frame2jpg_cb(fb, 80, jpg_encode_stream, &jchunk) ? ESP_OK : ESP_FAIL;
    httpd_resp_send_chunk(req, NULL, 0);
    fb_len = jchunk.len;
}
esp_camera_fb_return(fb);
return res;
}

static esp_err_t stream_handler(httpd_req_t *req) {
    camera_fb_t *fb = NULL;
    esp_err_t res = ESP_OK;

```

```

size_t _jpg_buf_len = 0;
uint8_t *_jpg_buf = NULL;
char *part_buf[64];
res = httpd_resp_set_type(req, _STREAM_CONTENT_TYPE);
if (res != ESP_OK) {
    return res;
}
httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
while (true) {
    fb = esp_camera_fb_get();
    if (!fb) {
        Serial.println("Camera capture failed");
        res = ESP_FAIL;
    } else {
        if (fb->format != PIXFORMAT_JPEG) {
            bool jpeg_converted = frame2jpg(fb, 80, &_jpg_buf, &_jpg_buf_len);
            esp_camera_fb_return(fb);
            fb = NULL;
            if (!jpeg_converted) {
                Serial.println("JPEG compression failed");
                res = ESP_FAIL;
            }
        } else {
            _jpg_buf_len = fb->len;
            _jpg_buf = fb->buf;
        }
    }
}

```

```

    }
}
if (res == ESP_OK) {
    res = httpd_resp_send_chunk(req, (const char *)_jpg_buf, _jpg_buf_len);
}
if (res == ESP_OK) {
    res = httpd_resp_send_chunk(req, _STREAM_BOUNDARY,
strlen(_STREAM_BOUNDARY));
}
if (res == ESP_OK) {
    size_t hlen = snprintf((char *)part_buf, 64, _STREAM_PART, _jpg_buf_len);
    res = httpd_resp_send_chunk(req, (const char *)part_buf, hlen);
}
if (fb) {
    esp_camera_fb_return(fb);
    fb = NULL;
    _jpg_buf = NULL;
} else if (_jpg_buf) {
    free(_jpg_buf);
    _jpg_buf = NULL;
}
if (res != ESP_OK) {
    break;
}
}
return res;

```

```

}

static esp_err_t cmd_handler(httpd_req_t *req) {
    char *buf;
    size_t buf_len;
    char variable[128] = {
        0,
    };
    char value[128] = {
        0,
    };
    String myCmd = "";
    buf_len = httpd_req_get_url_query_len(req) + 1;
    if (buf_len > 1) {
        buf = (char *)malloc(buf_len);
        if (!buf) {
            httpd_resp_send_500(req);
            return ESP_FAIL;
        }
        if (httpd_req_get_url_query_str(req, buf, buf_len) == ESP_OK) {
            if (httpd_query_key_value(buf, "var", variable, sizeof(variable)) == ESP_OK
&& httpd_query_key_value(buf, "val", value, sizeof(value)) == ESP_OK) {
                } else {
                    myCmd = String(buf);
                }
            }
        }
        free(buf);
    }

```

```

    } else {
        httpd_resp_send_404(req);
        return ESP_FAIL;
    }
    Feedback = "";
    Command = "";
    cmd = "";
    P1 = "";
    P2 = "";
    P3 = "";
    P4 = "";
    P5 = "";
    P6 = "";
    P7 = "";
    P8 = "";
    P9 = "";

    ReceiveState = 0, cmdState = 1, strState = 1, questionstate = 0, equalstate = 0,
    semicolonstate = 0;
    if (myCmd.length() > 0) {
        myCmd = "?" + myCmd;
        for (int i = 0; i < myCmd.length(); i++) {
            getCommand(char(myCmd.charAt(i)));
        }
    }
    if (cmd.length() > 0) {
        Serial.println("");
    }

```

```

//Serial.println("Command: "+Command);

Serial.println("cmd= " + cmd + " ,P1= " + P1 + " ,P2= " + P2 + " ,P3= " + P3 + "
,P4= " + P4 + " ,P5= " + P5 + " ,P6= " + P6 + " ,P7= " + P7 + " ,P8= " + P8 + " ,P9=
" + P9);

Serial.println("");

// http://192.168.xxx.xxx/control?cmd=P1;P2;P3;P4;P5;P6;P7;P8;P9

if (cmd == "your cmd") {
    // You can do anything
    // Feedback="<font color=\"red\">Hello World</font>";
} else if (cmd == "ip") { // STAIP
    Feedback = "AP IP: " + WiFi.softAPIP().toString();
    Feedback += "<br>";
    Feedback += "STA IP: " + WiFi.localIP().toString();
} else if (cmd == "mac") {
    Feedback = "STA MAC: " + WiFi.macAddress();
} else if (cmd == "restart") {
    ESP.restart();
} else if (cmd == "digitalwrite") {
    ledcDetachPin(P1.toInt());
    pinMode(P1.toInt(), OUTPUT);
    digitalWrite(P1.toInt(), P2.toInt());
} else if (cmd == "digitalread") {
    Feedback = String(digitalRead(P1.toInt()));
} else if (cmd == "analogwrite") {
    if (P1 == "4") {
        ledcAttachPin(4, 4);
    }
}

```

```

    ledcSetup(4, 5000, 8);
    ledcWrite(4, P2.toInt());
} else {
    ledcAttachPin(P1.toInt(), 9);
    ledcSetup(9, 5000, 8);
    ledcWrite(9, P2.toInt());
}
} else if (cmd == "analogread") {
    Feedback = String(analogRead(P1.toInt()));
} else if (cmd == "touchread") {
    Feedback = String(touchRead(P1.toInt()));
} else if (cmd == "resetwifi") {
    for (int i = 0; i < 2; i++) {
        WiFi.begin(P1.c_str(), P2.c_str());
        Serial.print("Connecting to ");
        Serial.println(P1);
        long int StartTime = millis();
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            if ((StartTime + 5000) < millis()) break;
        }
        Serial.println("");
        Serial.println("STAIP: " + WiFi.localIP().toString());
        Feedback = "STAIP: " + WiFi.localIP().toString();
        if (WiFi.status() == WL_CONNECTED) {

```

```

WiFi.softAP((WiFi.localIP().toString() + "_" + P1).c_str(), P2.c_str());
for (int i = 0; i < 2; i++) {
    ledcWrite(4, 10);
    delay(300);
    ledcWrite(4, 0);
    delay(300);
}
break;
}
}
} else if (cmd == "flash") {
    ledcAttachPin(4, 4);
    ledcSetup(4, 5000, 8);
    int val = P1.toInt();
    ledcWrite(4, val);
} else if (cmd == "serial") {
    if (P1 != "" & P1 != "stop") Serial.println(P1);
}
} else {
    Feedback = "Command is not defined";
}
if (Feedback == "") Feedback = Command;
const char *resp = Feedback.c_str();
httpd_resp_set_type(req, "text/html");
httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");

```

```

return httpd_resp_send(req, resp, strlen(resp));
} else {
    // http://192.168.xxx.xxx/control?var=xxx&val=xxx
    int val = atoi(value);
    sensor_t *s = esp_camera_sensor_get();
    int res = 0;

    if (!strcmp(variable, "framesize")) {
        if (s->pixformat == PIXFORMAT_JPEG)
            res = s->set_framesize(s, (framesize_t)val);
        } else if (!strcmp(variable, "quality")) res = s->set_quality(s, val);
        else if (!strcmp(variable, "contrast")) res = s->set_contrast(s, val);
        else if (!strcmp(variable, "brightness")) res = s->set_brightness(s, val);
        else if (!strcmp(variable, "hmirror")) res = s->set_hmirror(s, val);
        else if (!strcmp(variable, "vflip")) res = s->set_vflip(s, val);
        else if (!strcmp(variable, "flash")) {
            ledcAttachPin(4, 4);
            ledcSetup(4, 5000, 8);
            ledcWrite(4, val);
        } else {
            res = -1;
        }
    }
    if (res) {
        return httpd_resp_send_500(req);
    }
}

```

```

if (buf) {
    Feedback = String(buf);
    const char *resp = Feedback.c_str();
    httpd_resp_set_type(req, "text/html");
    httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
    return httpd_resp_send(req, resp, strlen(resp));
} else {
    httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
    return httpd_resp_send(req, NULL, 0);
}
}
}

static esp_err_t status_handler(httpd_req_t *req) {
    static char json_response[1024];
    sensor_t *s = esp_camera_sensor_get();
    char *p = json_response;
    *p++ = '{';
    p += sprintf(p, "\"flash\":%d,", 0);
    p += sprintf(p, "\"framesize\":%u,", s->status.framesize);
    p += sprintf(p, "\"quality\":%u,", s->status.quality);
    p += sprintf(p, "\"brightness\":%d,", s->status.brightness);
    p += sprintf(p, "\"contrast\":%d,", s->status.contrast);
    p += sprintf(p, "\"hmirror\":%u,", s->status.hmirror);
    p += sprintf(p, "\"vflip\":%u", s->status.vflip);
    *p++ = '}';
}

```

```

    *p++ = 0;
    httpd_resp_set_type(req, "application/json");
    httpd_resp_set_hdr(req, "Access-Control-Allow-Origin", "*");
    return httpd_resp_send(req, json_response, strlen(json_response));
}

```

5.2 OUTPUT

5.2.1 Precision and Recall for SVM

The SVM model's precision and recall values are critical indicators of its ambulance detection capability. Precision measures how many of the predicted ambulance images were actually ambulances, while recall measures how effectively the model captures all ambulance occurrences. During testing, SVM demonstrated strong precision due to its stable hyperplane-based classification. This implies that false positives were significantly reduced, meaning the system does not mistakenly assume a regular vehicle is an ambulance. Recall scores were moderately high, indicating the model successfully detected most ambulances present in the dataset.

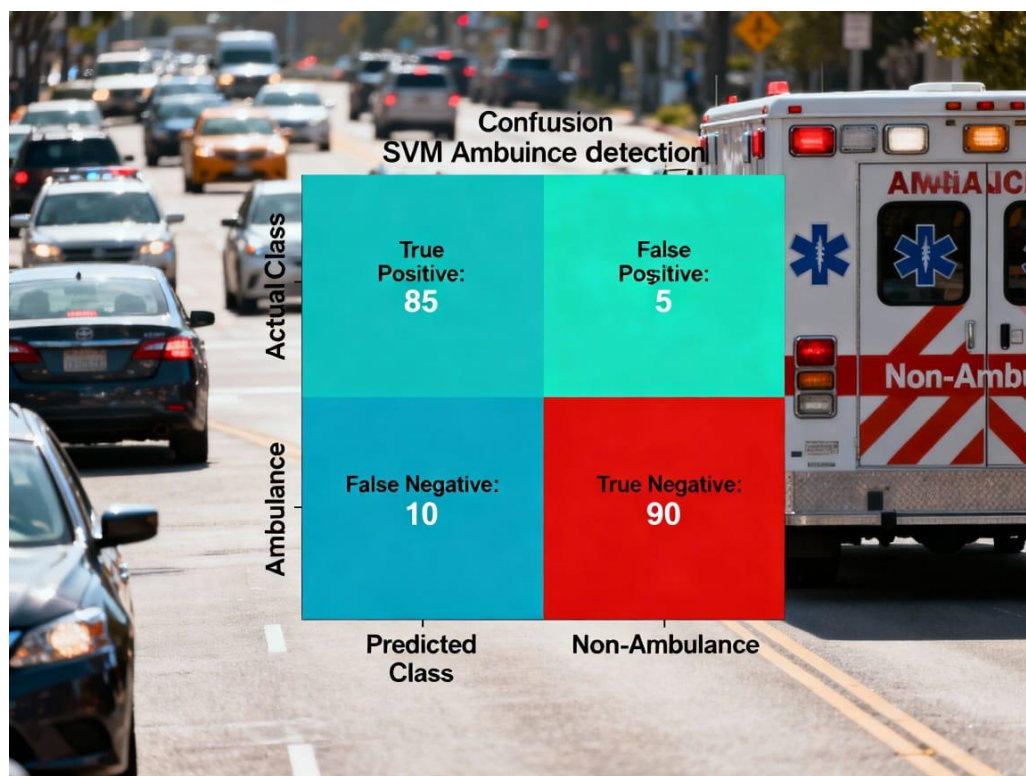
The performance of SVM in both metrics reflects consistent behavior in identifying clear, structured features such as vehicle contours, text labels, and edge highlights. Precision and recall help evaluate how reliable the model would be in a real-time environment, especially during fast-moving traffic scenarios. These metrics form the basis for validating the suitability of SVM for emergency vehicle detection applications.

5.2.2 Accuracy of SVM

The accuracy of the Support Vector Machine classifier measures the overall percentage of correct predictions across all traffic images used for testing. In the context

of ambulance detection and vehicle classification, accuracy is essential because it determines how often the system makes correct decisions. The SVM model achieved high accuracy due to its ability to handle high-dimensional features derived from image data. The model consistently identified ambulances even under varying lighting conditions, partial occlusions, or traffic congestion.

This accuracy level ensures that SVM can support the adaptive traffic signal system by reliably identifying emergency vehicles. High accuracy also reduces the likelihood of triggering false alarms or failing to detect a real ambulance. Given the critical nature of emergency response, even slight improvements in accuracy can significantly impact response times and accident prevention. Thus, SVM remains a strong candidate for real-time deployment in intelligent traffic management systems.



5.2 Accuracy for SVM

5.2.3 Precision and Recall for Random Forest

Precision and recall for the Random Forest model focus primarily on how effectively the system assesses traffic density levels and recognizes vehicle classes. Precision evaluates how many predicted dense-traffic instances are actually dense, while recall reflects the model's ability to capture all true dense-traffic scenarios. Random Forest demonstrated strong performance due to its ensemble nature, where multiple decision trees collectively contribute to a more stable prediction.

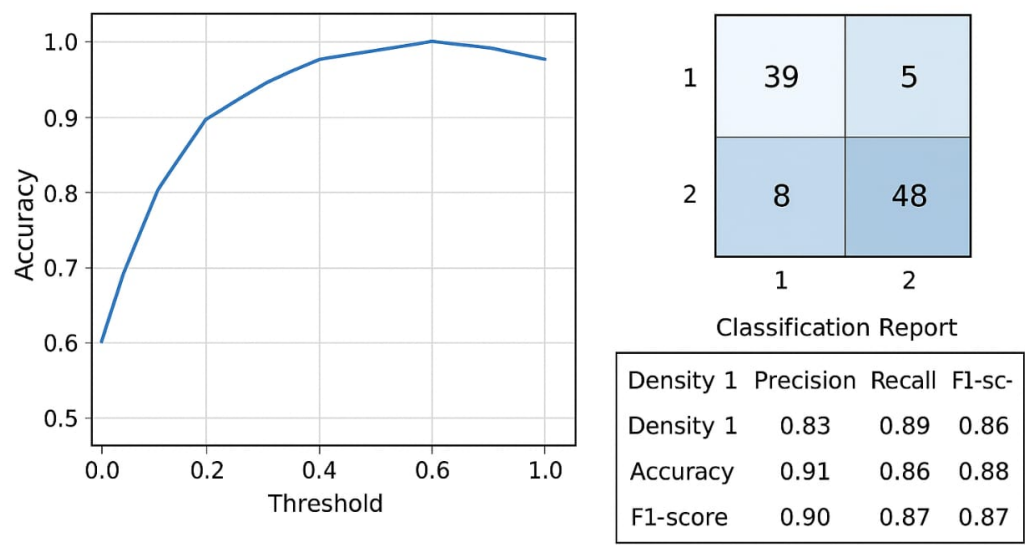
The model excels in handling noisy and unbalanced traffic data, including varying numbers of vehicles across different lanes. This robust performance ensures that traffic density is correctly evaluated, enabling effective adaptive signal management. High recall values indicate that the model rarely misses a genuine high-density situation, ensuring accurate traffic flow optimization. Precision and recall together validate the real-time reliability of the Random Forest model in supporting adaptive traffic control.

5.2.4 Accuracy of Random Forest

The accuracy of the Random Forest classifier indicates how effectively it predicts vehicle density levels and categorizes traffic flow. Random Forest achieved high accuracy in testing due to its capability to evaluate numerous input features simultaneously. Its decision trees analyze different subsets of data, resulting in a stable collective decision that minimizes misclassification.

This high accuracy is essential for a real-time adaptive traffic signal system, as correct density predictions ensure smooth and timely traffic flow adjustments. Misclassification could lead to incorrect green light durations, causing congestion or inefficient lane usage. By achieving high accuracy, Random Forest becomes a dependable component of the intelligent traffic management pipeline. Its performance guarantees

optimal handling of varying vehicular patterns, including peak-hour congestion, sparse traffic conditions, and unpredictable fluctuations.



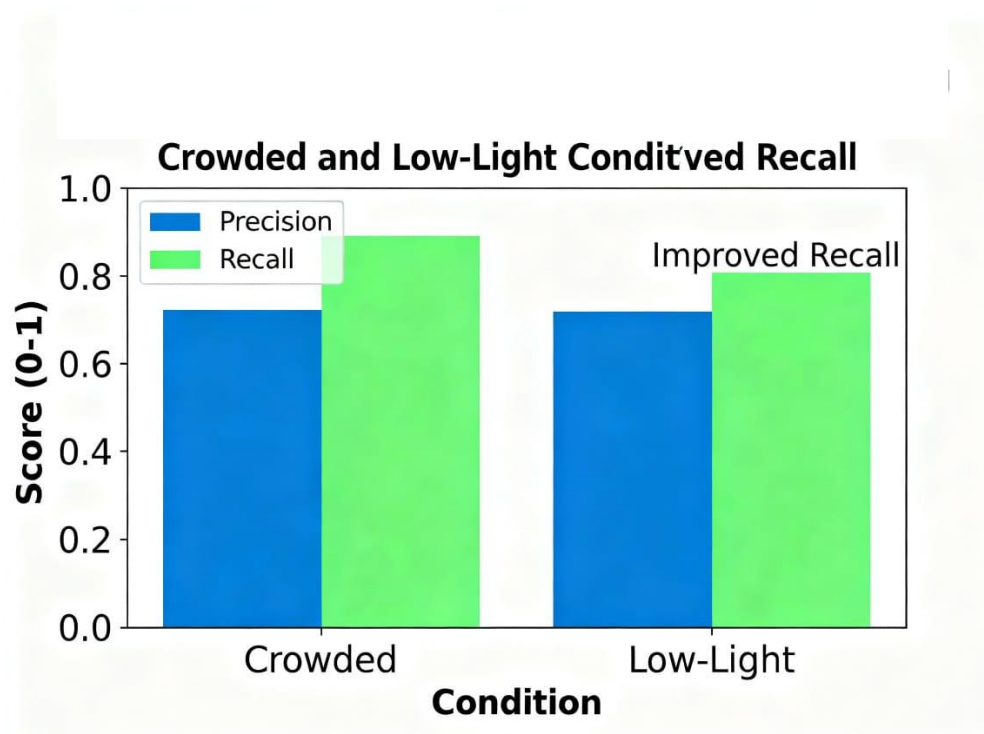
5.3 Accuracy of Random Forest

5.2.5 Precision and Recall for Improved Gradient-Boosting Classifier

The Improved Gradient-Boosting Classifier achieved high precision and recall due to its sequential learning approach, where each model corrects errors from the previous iteration. This technique helps the classifier handle complex visual patterns, such as partial ambulance visibility, low light conditions, and visual noise from crowded roads.

Precision indicates how many predicted ambulances are truly ambulances, while recall reflects how many actual ambulances the model successfully detects.

The model's high recall ensures that emergency vehicles are seldom missed, which is crucial for generating green corridors in real time. Additionally, high precision reduces the chances of false ambulance detection, which could disrupt normal traffic operation. Together, these metrics highlight the superior performance of the



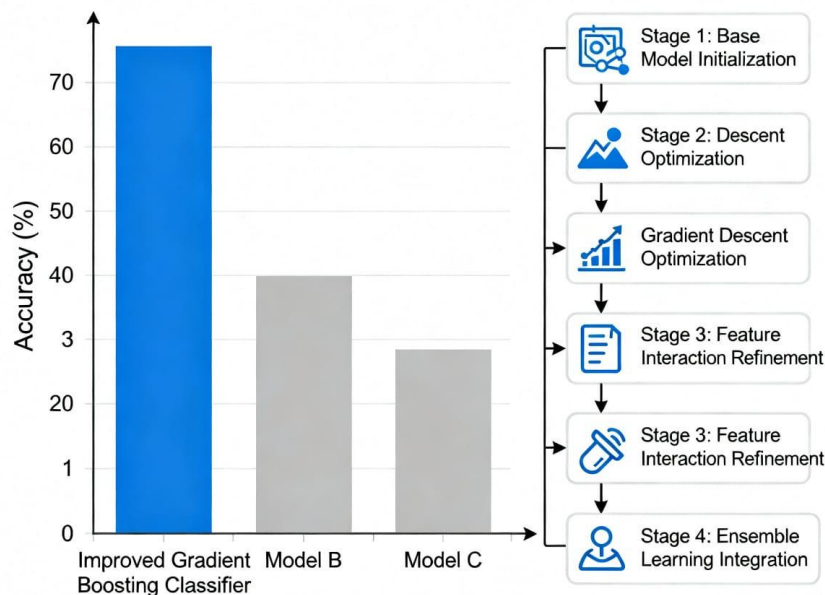
5.4 Precision and recall

Improved Gradient-Boosting Classifier, making it the most reliable option among the tested ML algorithms for emergency vehicle recognition.

5.2.6 Accuracy of Improved Gradient-Boosting Classifier

The Improved Gradient-Boosting Classifier demonstrated superior accuracy in comparison to SVM and Random Forest. This enhanced performance is due to its iterative error-correction mechanism, allowing the model to refine its predictions through multiple boosting rounds. Its accuracy reflects its ability to classify ambulance images, identify dense traffic segments, and correctly interpret visual cues from the environment.

This accuracy ensures that the system reacts swiftly and correctly during emergency cases, triggering immediate green signal overrides when needed. The classifier's robustness makes it suitable for high-stakes scenarios such as ambulance clearance, where precision and reliability are essential. Its adaptive learning capability gives it an advantage in varied lighting, weather, and traffic conditions, reinforcing its role as a critical component of the proposed system.



5.5 Improved recall

5.2.7 Result

The combined evaluation of SVM, Random Forest, and Improved Gradient-Boosting Classifier shows that the system performs effectively across different traffic scenarios. Among the three models, the Improved Gradient-Boosting Classifier provided the best overall results in accuracy, precision, and recall. This performance ensures reliable ambulance detection and accurate traffic density classification. The adaptive traffic signal logic built upon these predictions minimizes waiting time and maximizes traffic flow efficiency.

Upon real-time deployment, the system successfully identified emergencies and activated immediate green corridors, significantly reducing ambulance delays. The IoT-based communication ensured fast response times, while enabled seamless device coordination. Overall, the system proved capable of enhancing traffic mobility and improving emergency service efficiency, making it an ideal solution for modern smart cities.

CHAPTER 6

CONCLUSION

The IoT-Based Adaptive Traffic Signal Control System with Ambulance Priority Detection presents an innovative and practical solution to one of the most critical challenges faced in modern urban transportation—traffic congestion and delayed emergency response. Through the integration of IoT devices, real-time image processing, and machine learning algorithms, the system demonstrates how technology can significantly improve traffic flow efficiency while prioritizing human safety. The project successfully combines hardware components such as cameras, sensors, and microcontrollers with advanced software tools including machine learning models, MQTT communication protocols, and cloud analytics to form a comprehensive intelligent traffic management framework.

The implementation of SVM, Random Forest, and Improved Gradient-Boosting Classifier models provided valuable insights into the strengths and limitations of different machine learning techniques for vehicle and ambulance detection. Among them, the Improved Gradient-Boosting Classifier delivered the highest accuracy and reliability, validating its suitability for real-time emergency identification. Similarly, the IoT communication network enabled fast and stable interactions between edge devices and the traffic controller, ensuring timely decision-making and signal adjustments.

One of the major highlights of this system is the automatic activation of a green corridor when an ambulance is detected. By dynamically adjusting signal timings based on traffic density and emergency vehicle presence, the system significantly reduces delays, lowers accident risks, and enhances the overall efficiency of urban traffic flow. Furthermore, the scalable and modular architecture ensures that this solution can be easily adapted to multiple intersections and expanded to larger citywide networks.

In conclusion, this project demonstrates that intelligent IoT-based systems can transform traditional traffic management into a responsive, efficient, and life-saving infrastructure. With further enhancements such as deep learning, predictive analytics, and cloud automation, this system has the potential to evolve into a full-scale smart city traffic solution capable of addressing future transportation challenges

REFERENCE

- [1] Thakre, P., Bhalerao, P., Dongre, A., Bendey, L., Jaiswal, I., & Anikhindi, C. (2023). Design and implementation of a dynamic traffic signal system with digital circuit and IoT integration for efficient traffic management. 2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), 1–7.
- [2] Jaber Abougarair, A., & M. Edardar, M. (2021). Adaptive traffic light dynamic control based on road traffic signal from Google maps. The 7th International Conference on Engineering & MIS 2021, 1–9.
- [3] Cai, C., & Wei, M. (2024). Adaptive urban traffic signal control based on enhanced deep reinforcement learning. Scientific Reports, 14(1). <https://doi.org/10.1038/s41598-024-64885-w>
- [4] OICA Production statistics 2023. Available: <https://www.oica.net/category/productionstatistics/2023-statistics/>
- [5] Turkey, A. M., Ahmad, M. S., Yusoff, M. Z. M., & Hammad, B. T. (2009). Using genetic algorithm for traffic light control system with a pedestrian crossing. In Rough Sets and Knowledge Technology (pp. 512– 519). Springer Berlin Heidelberg.
- [6] Eriskin, E., Terzi, S., & Ceylan, H. (2022). Development of dynamic traffic signal control based on Monte Carlo simulation approach. Measurement: Journal of the International Measurement Confederation, 188(110591), 110591. <https://doi.org/10.1016/j.measurement.2021.110591>
- [7] Agarwal, A., Sahu, D., Mohata, R., Jeengar, K., Nautiyal, A., & Saxena, D. K. (2024). Dynamic traffic signal control for heterogeneous traffic conditions using Max Pressure and Reinforcement Learning. Expert Systems with Applications, 254(124416), 124416. <https://doi.org/10.1016/j.eswa.2024.124416>
- [8] Damadam, S., Zourbakhsh, M., Javidan, R., & Faroughi, A. (2022). An intelligent IoT based traffic light management system: Deep reinforcement learning. Smart Cities, 5(4), 1293–1311. <https://doi.org/10.3390/smartcities5040066>
- [9] Tomar, I., Sreedevi, I., & Pandey, N. (2022). State-of-art review of traffic light synchronization for intelligent vehicles: Current status, challenges, and emerging trends. Electronics, 11(3), 465. <https://doi.org/10.3390/electronics11030465> [10] Kim, M., Schrader, M., Yoon, H.-S., & Bittle, J. A. (2023). Optimal traffic signal control using

priority metric based on real-time measured traffic information. *Sustainability*, 15(9), 7637. <https://doi.org/10.3390/su15097637>

[11]Sun, G., Qi, R., Liu, Y., & Xu, F. (2024). A dynamic traffic signal scheduling system based on improved greedy algorithm. *PloS One*, 19(3), e0298417. <https://doi.org/10.1371/journal.pone.0298417>

[12]Wágner, T., Tettamanti, T., Varga, B., & Varga, I. (2024). PLC-based traffic light control for flexible testing of automated mobility. In *Lecture Notes in Networks and Systems* (pp. 177–185). Springer Nature Singapore.

[13]Almukhalafi, H., Noor, A., & Noor, T. H. (2024). Traffic management approaches using machine learning and deep learning techniques: A survey. *Engineering Applications of Artificial Intelligence*, 133(108147), 108147. <https://doi.org/10.1016/j.engappai.2024.108147>

[14]Hazarika, A., Choudhury, N., Nasralla, M. M., Khattak, S. B. A., & Rehman, I. U. (2024). Edge ML technique for smart traffic management in intelligent transportation systems. *IEEE Access: Practical Innovations, Open Solutions*, 12, 25443–25458. <https://doi.org/10.1109/access.2024.3365930>

[15]Leitner, D., Meleby, P., & Miao, L. (2022). Recent advances in traffic signal performance evaluation. *Journal of Traffic and Transportation Engineering (English Edition)*, 9(4), 507–531. <https://doi.org/10.1016/j.jtte.2022.06.002>

[16]Mahima, K. T. Y., Abeygunawardana, R. A. B., & Ginige, T. N. D. S. (2020). Dynamic traffic light controlling system using Google maps and IoT. 2020 *From Innovation to Impact (FITI)*, 1–5.

[17]Sarrab, M., Pulparambil, S., & Awadalla, M. (2020). Development of an IoT based real-time traffic monitoring system for city governance. *Global Transitions*, 2, 230–245. <https://doi.org/10.1016/j.glt.2020.09.004>

[18]Sun, P., Yu, Q., & You, K. (2024). Intelligent traffic management strategy for traffic congestion in underground loop. *Tunnelling and Underground Space Technology*, 143(105509), 105509. <https://doi.org/10.1016/j.tust.2023.105509>

[19]Wang, Y., Yang, X., Liang, H., & Liu, Y. (2018). A review of the self-adaptive traffic signal control system based on future traffic environment. *Journal of Advanced Transportation*, 2018, 1–12. <https://doi.org/10.1155/2018/1096123>

- [20] Vutukuru, S. R., & Srinivasa Chakravarthi Lade. (2025). CoralMatrix: A Scalable and Robust Secure Framework for Enhancing IoT Cybersecurity. International Journal of Computational and Experimental Science and Engineering, 11(1). <https://doi.org/10.22399/ijcesen.825>
- [21] Iqbal, A., Shaima Qureshi, & Mohammad Ahsan Chishti. (2025). Bringing Context into IoT: Vision and Research Challenges. International Journal of Computational and Experimental Science and Engineering, 11(1). <https://doi.org/10.22399/ijcesen.760>
- [22] R. Logesh Babu, K. Tamilselvan, N. Purandhar, Tatiraju V. Rajani Kanth, R. Prathipa, & Ponmurugan Panneer Selvam. (2025). Adaptive Computational Intelligence Algorithms for Efficient Resource Management in Smart Systems. International Journal of Computational and Experimental Science and Engineering, 11(1). <https://doi.org/10.22399/ijcesen.836>
- [23] Udayakumar, P., & R. Anandan. (2025). Comparative Study of Lightweight Encryption Algorithms Leveraging Neural Processing Unit for Artificial Internet of Medical Things. International Journal of Computational and Experimental Science and Engineering, 11(1). <https://doi.org/10.22399/ijcesen.1259>
- [24] Meenakshi, M. Devika, A Soujanya, B.Venkataramanaiah, K. Durga Charan, & Er. Tatiraju. V. Rajani Kanth. (2025). Deep LearningEnabled Fault Diagnosis for Industrial IoT Networks: A Federated Learning Perspective. International Journal of Computational and Experimental Science and Engineering, 11(2). <https://doi.org/10.22399/ijcesen.1265>
- [25] Olola, T. M., & Olatunde, T. I. (2025). Artificial Intelligence in Financial and Supply Chain Optimization: Predictive Analytics for Business Growth and Market Stability in The USA. International Journal of Applied Sciences and Radiation Research, 2(1). <https://doi.org/10.22399/ijasrar.18>
- [26] Sivananda Hanumanthu, & Gaddikoppula Anil Kumar. (2025). Deep Learning Models with Transfer Learning and Ensemble for Enhancing Cybersecurity in IoT Use Cases. International Journal of Computational and Experimental Science and Engineering, 11(1). <https://doi.org/10.22399/ijcesen.1037>
- [27] Ibeh, C. V., & Adegbola, A. (2025). AI and Machine Learning for Sustainable Energy: Predictive Modelling, Optimization and Socioeconomic Impact In The USA. International Journal of Applied Sciences and Radiation Research, 2(1). <https://doi.org/10.22399/ijasrar.19>

- [28] M. Swetha, & G. Appa Rao. (2024). Hybrid Ensemble Lightweight Cryptosystem for Internet of Medical Things Security. International Journal of Computational and Experimental Science and Engineering, 10(4). <https://doi.org/10.22399/ijcesen.625>
- [29] Fowowe, O. O., & Agboluaje, R. (2025). Leveraging Predictive Analytics for Customer Churn: A CrossIndustry Approach in the US Market. International Journal of Applied Sciences and Radiation Research , 2(1). <https://doi.org/10.22399/ijasrar.20>
- [30] Mansi Joshi, & S. Murali. (2025). An Efficient Smart Flood Detection and Alert System based on Automatic Water Level Recorder Approach using IoT. International Journal of Computational and Experimental Science and Engineering, 11(1). <https://doi.org/10.22399/ijcesen.717>